



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

TRABAJO DE FIN DE GRADO

David Vázquez Rivas

Grado en Ingeniería Informática

Escola Politècnica Superior

2025-2026

TRABAJO DE FIN DE GRADO

David Vázquez Rivas

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

2025-2026

Paraules clau del treball: Gestión de Proyectos

Tutor: Raimon Jaume Massant Vila

ÍNDICE GENERAL

Índice general	i
1 Acrónimos	1
2 Introducción	3
2.1 Contexto	3
2.2 Motivación	3
2.3 Objetivos	4
2.3.1 Objetivo General	4
2.3.2 Objetivos Específicos	4
3 Análisis	5
3.1 Interesados	5
3.1.1 Usuarios	5
3.1.2 Stakeholders?	5
3.2 Especificación funcional	5
3.3 Especificación no funcional	9
3.4 Modelado del Dominio	11
3.4.1 Lenguaje ubicuo	11
3.4.2 Modelo de Datos	13
3.4.3 Modelo de Usuarios y Identidad	13
3.4.4 Módulo de Organizaciones	14
3.4.5 Módulo de Proyectos	15
3.4.6 Módulo de Elementos de Trabajo	17
3.4.7 Módulo de Chats	19
3.4.8 Módulo de Agentes	20
3.4.9 Modelo de Notificaciones	22
4 Diseño	23
4.1 Arquitectura de la aplicación	23
4.1.1 Diagrama de contexto (Nivel 1)	23
4.1.2 Diagrama de contenedores (Nivel 2)	23
4.1.3 Diagrama de componentes (Nivel 3)	23
5 Planificación	25
5.1 Alcance del proyecto	25
5.1.1 Definición del alcance	25

5.1.2	Exclusiones	25
5.2	Cronograma	26
A	Anexos	27
	Bibliografía	29

CAPÍTULO 1

ACRÓNIMOS

IA Inteligencia Artificial

UX Experiencia de Usuario

MVP Producto Mínimo Viable

PM Project Manager

CI Integración Continua

DDD Domain-Driven Design

MER Modelo Entidad-Relación

CQRS Command Query Responsibility Segregation

API Interfaz de Programación de Aplicaciones

INTRODUCCIÓN

2.1 Contexto

TODO

2.2 Motivación

Aunque las herramientas actuales de gestión han integrado con éxito visualizaciones clásicas, estas en su mayoría perpetúan paradigmas analógicos. Esta herencia visual desaprovecha las capacidades de las interfaces modernas para sintetizar la complejidad, lo que genera dos barreras principales en la industria actual:

- **Alta carga cognitiva y fatiga visual:** Para comprender el estado real de un proyecto, los gestores y desarrolladores deben procesar manualmente una gran cantidad de texto y datos abstractos dispersos. La falta de representaciones visuales intuitivas dificulta la detección rápida de bloqueos, dependencias o riesgos, obligando al usuario a realizar un esfuerzo mental considerable para construir un modelo mental del proyecto.
- **Burocracia frente a trabajo de valor:** Gran parte del tiempo de gestión se invierte en tareas mecánicas y repetitivas: solicitar actualizaciones de estado, requerir la cumplimentación de campos y sintetizar información fragmentada. Esta microgestión interrumpe los estados de concentración de los perfiles técnicos y desvía a los responsables de la toma de decisiones estratégicas.

2.3 Objetivos

2.3.1 Objetivo General

Investigar, diseñar y desarrollar un nuevo paradigma de plataforma de gestión de proyectos que reemplace las vistas tabulares convencionales por metáforas visuales inmersivas, capaces de representar la complejidad estructural del trabajo. Asimismo, se pretende concebir e integrar un sistema inteligente capaz de actuar de forma autónoma para asistir en la coordinación asíncrona y el seguimiento de los equipos.

2.3.2 Objetivos Específicos

Para alcanzar el objetivo general, se plantean los siguientes objetivos específicos:

- **A. Investigación y análisis:** Analizar las limitaciones de usabilidad y Experiencia de Usuario (UX) en las herramientas de gestión convencionales y estudiar su impacto en la carga cognitiva. Investigar patrones de diseño de interfaces en sectores ajenos a la gestión empresarial que permitan representar jerarquías y dependencias de forma intuitiva.
- **B. Diseño conceptual y visual:** Definir una propuesta de interfaz gráfica y experiencia de usuario que permita visualizar la topología de un proyecto como un sistema interconectado. El objetivo es priorizar la claridad visual frente a la lectura extensiva de texto, fomentando una interacción más natural y adaptada a entornos profesionales.
- **C. Sistema de agentes:** Diseñar un sistema basado en Inteligencia Artificial (IA) capaz de actuar como intermediario ágil entre el usuario y la plataforma. El propósito es establecer flujos de trabajo que permitan delegar tareas repetitivas de coordinación, comunicación asíncrona y seguimiento de estados.
- **D. Arquitectura de software:** Diseñar e implementar una base técnica robusta que garantice la escalabilidad, el alto rendimiento y la mantenibilidad del código. Se persigue estructurar el sistema aplicando principios de diseño de software avanzados que aseguren un alto grado de desacoplamiento entre los dominios de la aplicación, facilitando su evolución y mantenimiento futuros.
- **E. Prototipado y validación técnica:** Desarrollar un Producto Mínimo Viable (MVP) que integre los nuevos paradigmas de visualización y el sistema multi-agente. Validar la viabilidad de la arquitectura propuesta y evaluar si la solución aporta un valor funcional tangible frente a los enfoques de gestión de proyectos tradicionales.

ANÁLISIS

3.1 Interesados

3.1.1 Usuarios

Se han identificado dos tipos de usuarios principales para la aplicación: los gestores de proyectos y los miembros del equipo de proyecto. Ambos tipos de usuarios son agrupados bajo el término genérico de "usuario", ya que ambos interactúan con la aplicación, aunque con diferentes objetivos y necesidades.

Gestores: Son los usuarios encargados de la dirección y supervisión del proyecto. Tienen visión global del proyecto. A nivel de rol, puede ser un Project Manager (PM) tradicional, pero también puede ser un rol de liderazgo técnico (Tech Lead, Product Owner, Arquitecto de Software, etc.) que asuma responsabilidades de gestión de proyecto.

Miembros del equipo: Son los usuarios encargados de la ejecución de las tareas del proyecto. Su interés principal en la aplicación es que el overhead de gestión sea mínimo, y que les permita centrarse en su trabajo. A nivel de rol, pueden ser desarrolladores, diseñadores, analistas, incluso agentes de IA que colaboren en el proyecto.

3.1.2 Stakeholders?

3.2 Especificación funcional

La especificación de los requisitos funcionales se ha realizado con la técnica de historias de usuario, que permite describir las funcionalidades de la aplicación desde la perspectiva del usuario final. Cada historia de usuario se ha formulado siguiendo el formato:

“Como [*tipo de usuario*], quiero [*funcionalidad*] para [*beneficio*].”

3. ANÁLISIS

Además, cada historia de usuario se ha priorizado utilizando la técnica MoSCoW, que clasifica las funcionalidades en Must Have (imprescindibles), Should Have (deseables), Could Have (interesantes) y Won't Have (no necesarias).

También se han organizado las historias de usuario en *épicas*, que agrupan funcionalidades relacionadas entre sí.

Cuadro 3.1: Historias de usuario

ID	Historia	Prioridad
Épica 1: Identidad y acceso		
HU-01	Como usuario, quiero registrarme mediante OAuth (Google, Microsoft, Facebook, X) para acceder a mi espacio de trabajo de forma segura sin gestionar nuevas contraseñas.	Must Have
Épica 2: Organización		
HU-02	Como gestor, quiero crear una organización que agrupe proyectos y usuarios para centralizar la gestión de mi empresa o grupo de trabajo en un solo lugar.	Should Have
HU-03	Como gestor, quiero definir roles y permisos de la organización para permitir a ciertos usuarios diferentes acciones sobre los proyectos de la organización.	Should Have
Épica 3: Configuración de proyectos		
HU-04	Como gestor, quiero crear proyectos dentro de una organización o de forma independiente para organizar el trabajo en diferentes iniciativas o clientes.	Must Have
HU-05	Como gestor, quiero poder configurar nombre, abreviación y privacidad de los proyectos para gestionar el acceso y la visibilidad de la información del proyecto.	Must Have
HU-06	Como gestor, quiero definir permisos y roles a nivel de proyecto para controlar el acceso a la información y funcionalidades del proyecto.	Must Have
HU-07	Como gestor, quiero invitar a usuarios a un proyecto para colaborar con mi equipo en la gestión del proyecto.	Must Have
HU-08	Como gestor, quiero clonar un proyecto existente para reutilizar la estructura y configuración de un proyecto anterior como plantilla en un nuevo proyecto.	Could Have
HU-09	Como gestor, quiero añadir campos personalizados a los elementos de trabajo del proyecto para adaptar la gestión del proyecto a las necesidades específicas de mi equipo o proyecto.	Should Have
Épica 4: Elementos de Trabajo		
Continúa en la siguiente página		

Tabla 3.1 – continuación		
ID	Historia	Prioridad
HU-10	Como usuario, quiero crear y gestionar elementos de trabajo para organizar y seguir el progreso de las tareas dentro de un proyecto.	Must Have
HU-11	Como usuario, quiero que los elementos de trabajo tengan un set de campos predefinidos (título, descripción, estado, responsable, etiquetas, etc.) para facilitar la organización y seguimiento de las tareas dentro de un proyecto.	Must Have
HU-12	Como usuario, quiero poder dividir el trabajo en elemento de trabajo más pequeños (subtareas) para gestionar tareas complejas dividiéndolas en partes más manejables y asignar diferentes responsables a cada parte.	Must Have
HU-13	Como miembro del equipo, quiero poder añadir comentarios a los elementos de trabajo y ver un historial de cambios para facilitar la comunicación y colaboración entre los miembros del equipo, y mantener un registro de las decisiones relacionadas con cada tarea.	Should Have
Épica 5: Notificaciones		
HU-14	Como usuario, quiero recibir notificaciones sobre eventos que me afecten (asignación de tareas, cambios en tareas asignadas, menciones, etc.) para mantenerme informado sobre el progreso del proyecto y las tareas que me afectan.	Should Have
HU-15	Como usuario, quiero recibir notificaciones cuando se hace una solicitud de aprobación de una acción que requiera mi aprobación para poder desbloquear el trabajo de otros miembros del equipo.	Should Have
Épica 6: Chats y agentes		
HU-16	Como usuario, quiero poder tener chats, tanto con otros usuarios como con agentes de IA para facilitar la comunicación y colaboración entre los miembros del equipo.	Should Have
HU-17	Como usuario, quiero poder crear agentes de IA que me ayuden en la gestión del proyecto y tengan un rol específico para automatizar tareas repetitivas, obtener resúmenes de información relevante, o recibir sugerencias para la gestión del proyecto.	Should Have
HU-18	Como gestor, quiero que los agentes de IA puedan realizar acciones dentro de la aplicación (crear elementos de trabajo, asignar tareas, enviar notificaciones, etc.) para automatizar tareas y procesos dentro de la gestión del proyecto.	Should Have
Continúa en la siguiente página		

3. ANÁLISIS

Tabla 3.1 – continuación		
ID	Historia	Prioridad
HU-19	Como gestor, quiero tener un agente de IA predefinido <i>Daily Standup</i> para recibir un resumen diario del estado del proyecto, incluyendo tareas pendientes, bloqueos, y progreso general.	Could Have
HU-20	Como gestor, quiero tener un agente de IA predefinido <i>Desglose de Trabajo</i> para poder introducir una tarea compleja y recibir una propuesta de desglose en subtareas más manejables.	Could Have
Épica 7: Visualización en Árbol		
Tras investigar diferentes opciones para mostrar el estado del proyecto de forma visual, se ha decidido implementar una vista en árbol que permita visualizar la estructura del proyecto y el progreso de cada elemento.		
HU-21	Como usuario, quiero visualizar el proyecto como un grafo donde los elementos de trabajo son nodos y las dependencias son conexiones para entender la estructura jerárquica y el orden de ejecución sin entrar al detalle de cada elemento de trabajo.	Must Have
HU-22	Como usuario, quiero identificar el tipo de trabajo por la forma del nodo y su esfuerzo estimado por el tamaño para distinguir rápidamente <i>features</i> complejas de tareas pequeñas sin leer.	Must Have
HU-23	Como usuario, quiero identificar el avance de cada elemento de trabajo por el relleno del nodo para conocer rápidamente el progreso.	Must Have
HU-24	Como usuario, quiero ver visualmente cuanto esfuerzo se ha dedicado a un elemento de trabajo respecto a lo planificado para detectar rápidamente desviaciones y poder actuar en consecuencia.	Must Have
HU-25	Como usuario, quiero ver en que estado se encuentra un elemento de trabajo mediante su color para coordinarme con el equipo y saber rápidamente si un elemento de trabajo está bloqueado, en progreso, o completado.	Must Have
HU-26	Como usuario, quiero ver quién tiene asignado un elemento de trabajo para coordinarme con el equipo.	Must Have
HU-27	Como usuario, quiero poder minimizar partes del árbol de trabajo para enfocarme en las partes del proyecto que me interesen sin perder la visión global.	Must Have
Épica 8: Perfil		
Continúa en la siguiente página		

Tabla 3.1 – continuación		
ID	Historia	Prioridad
HU-28	Como usuario, quiero tener un perfil personal indicando mi rol y zona horaria para que otros usuarios puedan conocer mi rol en los proyectos y para que las notificaciones se ajusten a mi zona horaria.	Could Have
HU-29	Como usuario, quiero acumular experiencia en diferentes áreas para que gestores y agentes de IA puedan recomendarme tareas o proyectos en función de mi experiencia y habilidades.	Could Have
HU-30	Como usuario, quiero que mi perfil contenga un indicador de calidad técnica para que los gestores y agentes de IA puedan tener en cuenta no solo mi experiencia, sino también la calidad de mis contribuciones.	Could Have
HU-31	Como usuario, quiero que mi perfil contenga un indicador de la velocidad a la que suelo completar tareas para que los gestores y agentes de IA puedan asignarme tareas teniendo en cuenta no solo mi experiencia y calidad, sino también mi velocidad de trabajo.	Could Have
HU-32	Como usuario, quiero que mi perfil contenga un indicador de mi sesgo de optimismo/pesimismo en la estimación de tareas para que los gestores y agentes de IA puedan ajustar las estimaciones de las tareas que me asignen teniendo en cuenta mi sesgo de optimismo/pesimismo.	Could Have

3.3 Especificación no funcional

Así mismo, se han identificado los requisitos no funcionales que rigen las restricciones, cualidades y atributos de calidad globales del sistema. Estos requisitos son críticos para garantizar el correcto funcionamiento, la robustez y la viabilidad práctica de la aplicación, abarcando dimensiones esenciales como la seguridad, el rendimiento, la escalabilidad, la disponibilidad, la mantenibilidad y la usabilidad.

Seguridad:

- **Autenticación estandarizada:** El control de acceso a la plataforma debe gestionarse mediante un sistema de autenticación seguro basado en los estándares industriales OAuth 2.0[1].
- **Control de acceso granular:** La protección de los datos e información sensible de las organizaciones y proyectos debe garantizarse mediante un modelo de control de acceso basado en roles (RBAC), aplicando restricciones jerárquicas tanto a nivel global de la organización como a nivel específico de cada proyecto.
- **Gestión de sesiones:** Las sesiones de los usuarios deben expirar automáticamente tras un periodo preconfigurado de inactividad y ofrecer la capacidad de revocar

de forma remota e instantánea cualquier sesión activa en caso de compromiso de la cuenta.

Rendimiento:

- **Tiempo de respuesta de la API:** El tiempo medio de respuesta del servidor para operaciones síncronas de lectura y escritura de datos debe mantenerse por debajo de los 300 ms para el 95 % de las solicitudes (percentil 95) bajo condiciones normales de carga.
- **Procesamiento asíncrono de IA:** Las tareas con alta carga computacional, tales como la generación de resúmenes por parte de agentes de IA o el desglose automatizado de elementos de trabajo complejos, deben procesarse de manera asíncrona, notificando al usuario el inicio del proceso.
- **Fluidez de la interfaz (Vista en árbol):** El motor de renderizado en el cliente debe asegurar una tasa de refresco fluida y cercana a los 60 fotogramas por segundo (fps) durante la interacción visual y la manipulación del grafo de tareas, minimizando el impacto del re-renderizado en proyectos complejos de hasta 500 elementos concurrentes.

Escalabilidad:

- **Escalado horizontal:** La arquitectura del sistema debe permitir el escalado horizontal independiente de sus componentes (servicios de aplicación, pasarelas de agentes de IA y bases de datos), facilitando el aislamiento de cargas de trabajo intensivas sin degradar el núcleo de la aplicación.
- **Concurrencia del sistema:** El sistema debe ser capaz de manejar de forma eficiente hasta 1000 usuarios concurrentes activos y soportar picos de carga de hasta 100 operaciones por segundo sin experimentar una degradación perceptible en los tiempos de respuesta.

Disponibilidad:

- **Tiempo de actividad (Uptime):** La aplicación debe garantizar una disponibilidad continua del 99.9 % del tiempo, excluyendo exclusivamente las ventanas de tiempo previamente notificadas para tareas de mantenimiento programado.
- **Tolerancia a fallos:** El sistema debe implementar mecanismos de recuperación automática y aislamiento de fallos, tales como políticas de reintentos, para mitigar caídas de servicios externos o de las APIs de proveedores de IA.

Mantenibilidad:

- **Arquitectura desacoplada:** El diseño del código fuente debe seguir un patrón arquitectónico modular, desacoplado y de alta cohesión (como la arquitectura hexagonal o limpia[2]), separando estrictamente la lógica del dominio de los detalles de infraestructura para facilitar su comprensión, mantenimiento y extensión.

- **Cobertura de pruebas:** Las capas de la aplicación que albergan las reglas de negocio y los casos de uso centrales deben contar con una cobertura de pruebas unitarias automatizadas de al menos un 80 %.
- **Inyección de configuración:** Todos los parámetros operativos, variables de entorno, credenciales de servicios externos e hiperparámetros de los modelos de IA deben estar completamente externalizados del código fuente, permitiendo modificaciones en caliente sin necesidad de recompilar o redesplegar la aplicación.
- **Análisis estático obligatorio:** El flujo de integración continua (Integración Continua (CI)) debe requerir la superación obligatoria de un análisis estático de código para garantizar los estándares de calidad del software, el control de la complejidad ciclomática y la ausencia de deuda técnica crítica antes de cada despliegue.

Usabilidad:

- **Heurísticas de diseño:** La interfaz de usuario deberá cumplir con los criterios de usabilidad validados mediante una evaluación heurística basada en los 10 principios de Nielsen y Molich [3].
- **Accesibilidad web:** La aplicación debe observar las directrices esenciales de accesibilidad web, asegurando relaciones de contraste cromático adecuadas en los nodos del árbol de tareas y el soporte para la navegación y operación mediante teclado en los formularios principales [4].

3.4 Modelado del Dominio

El análisis del núcleo de Leaftask se ha abordado desde la perspectiva del Domain-Driven Design (DDD) [5], priorizando la comprensión de las reglas de negocio. Dado que la gestión de proyectos asistida por inteligencia artificial presenta una alta complejidad conceptual, el dominio principal se ha descompuesto en múltiples *Bounded Contexts* [6].

En esta fase de análisis, cada *Bounded Context* [6] establece un límite estricto donde un subconjunto del modelo de dominio tiene total validez y autonomía. Esta separación permite aislar y entender áreas de responsabilidad claras (como la gestión de identidades, la estructura de los elementos de trabajo o la coordinación de agentes automatizados), garantizando que el modelo refleje fielmente la realidad del negocio antes de tomar decisiones.

A continuación, se detallan los pilares de este análisis, comenzando por el vocabulario compartido que vertebra el sistema, hasta llegar a la representación de los datos de cada contexto.

3.4.1 Lenguaje ubicuo

Siguiendo los principios del DDD[5], se ha definido un lenguaje ubicuo que unifica la terminología utilizada por los desarrolladores y los usuarios, facilitando la comunicación y el entendimiento mutuo entre ambos grupos. Este lenguaje se ha construido a

3. ANÁLISIS

partir de los términos y conceptos más relevantes para la gestión de proyectos, y se ha utilizado de forma consistente en toda la documentación, los diagramas y el código fuente de la aplicación.

Organización: Entidad principal que agrupa a múltiples usuarios y proyectos bajo un mismo espacio de trabajo, centralizando la gestión de permisos y roles a nivel corporativo o de grupo.

Proyecto: Iniciativa de trabajo acotada que contiene un conjunto estructurado de elementos de trabajo orientados a un objetivo común.

Miembro del Equipo: Entidad asignada a un proyecto para colaborar en su ejecución. Un miembro del equipo puede ser tanto un usuario humano (desarrollador, diseñador) como un Agente de IA.

Gestor: Rol encargado de la configuración, supervisión y administración de proyectos y organizaciones.

Agente de IA: Miembro del equipo automatizado que colabora en el proyecto. Actúa bajo un rol específico y tiene capacidad para interactuar con el sistema de manera análoga a un usuario humano.

Elemento de Trabajo (*Work Item*): Unidad básica de gestión dentro de un proyecto. Representa una tarea, funcionalidad o incidencia, y encapsula información como estado, responsable, esfuerzo estimado y dependencias.

Subtarea: Elemento de trabajo que se deriva o desglosa de un elemento de trabajo padre más complejo, estableciendo una relación jerárquica.

Árbol de Trabajo: Representación visual, jerárquica e interactiva del conjunto de elementos de trabajo de un proyecto en forma de grafo.

Nodo: Representación gráfica de un único Elemento de Trabajo dentro del Árbol de Trabajo. Sus atributos visuales (forma, tamaño, color y relleno) comunican instantáneamente el tipo de trabajo, el esfuerzo estimado, el estado y el avance.

Estado: Situación operativa actual de un elemento de trabajo en su ciclo de vida (pendiente, en progreso, bloqueado, completado).

Rol: Conjunto de responsabilidades y nivel de autoridad asignado a un Miembro del Equipo dentro de una Organización o Proyecto. Define qué acciones puede realizar en el sistema.

Permiso: Autorización explícita y granular para ejecutar una acción específica o acceder a un recurso concreto (ej. crear proyecto, invitar usuario, modificar elemento de trabajo). Los permisos se agrupan conformando un Rol.

Campo Personalizado: Atributo dinámico definido para extender la información predefinida de los Elementos de Trabajo, adaptándolos a las necesidades específicas de un proyecto.

Aprobación: Petición formal generada dentro del ciclo de vida de una acción que requiere la validación explícita de un usuario con los permisos adecuados para desbloquear el progreso.

3.4.2 Modelo de Datos

Siguiendo el enfoque de DDD[5], se ha diseñado un modelo conceptual de datos que refleja fielmente las entidades y relaciones del negocio. Si bien el dominio se concibe bajo un paradigma orientado a objetos, se ha optado por emplear el Modelo Entidad-Relación (MER)[7] para representar la estructura de la información que deberá ser persistente.

Dada la amplitud de la información que maneja el sistema, el modelo de datos no se presenta mediante un diagrama global. En su lugar, se ha segmentado haciendo corresponder los diagramas MER con los *Bounded Contexts* previamente identificados. Esta agrupación lógica agiliza la comprensión de los datos y asegura que cada módulo analizado mantenga una alta cohesión en torno a su contexto de negocio específico.

3.4.3 Modelo de Usuarios y Identidad

Este módulo es responsable de gestionar la información relacionada con los usuarios, su autenticación y sus estadísticas. Incluye las entidades:

- ***refresh_tokens***: Almacena los tokens de refresco generados para los usuarios, es importante persistirlos para poder invalidarlos en caso de compromiso de la cuenta.
- ***users***: Almacena la información básica de los usuarios, como su nombre, correo electrónico, rol y zona horaria.
- ***profile_images***: Almacena información sobre las imágenes de perfil de los usuarios como su URL y metadatos asociados.
- ***skill_types***: Almacena los tipos de habilidades que puede tener un usuario (lenguajes, metodologías, herramientas, etc.)
- ***skills***: Almacena las habilidades disponibles en el sistema, asociadas a un tipo de habilidad.
- ***user_skill_metrics***: Tabla intermedia que asocia a cada usuario con sus habilidades y contiene métricas como experiencia, calidad, velocidad y sesgo de predicción.

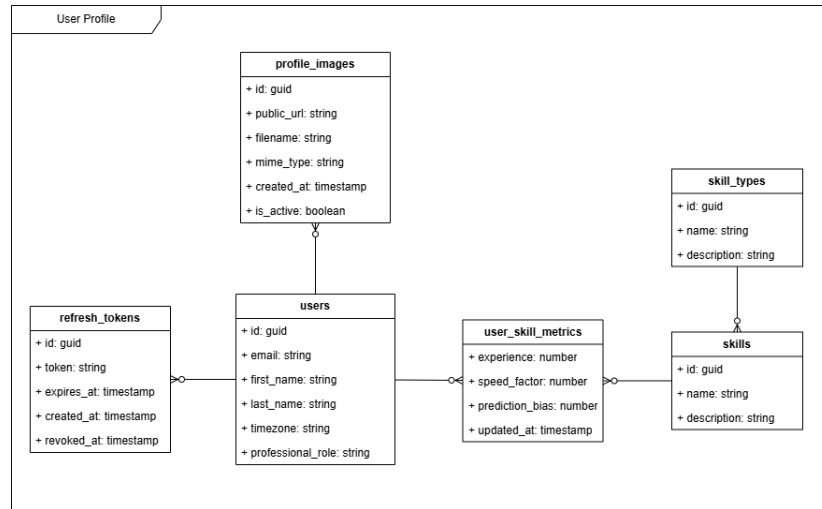


Figura 3.1: MER del módulo de Usuarios e Identidad

3.4.4 Módulo de Organizaciones

Este módulo gestiona la información relacionada con las organizaciones, sus miembros y roles. Incluye las entidades:

- **organizations**: Almacena la información básica de las organizaciones, como su nombre, descripción y website.
- **organization_invitations**: Almacena las invitaciones enviadas a usuarios para unirse a una organización, incluyendo el estado de la invitación. Los miembros activos de la organización, son los que tienen una invitación en estado aceptada.
- **organization_roles**: Almacena los roles definidos a nivel de organización, como Administrador, Gestor, Miembro, etc.
- **organization_permissions**: Almacena los permisos de organización disponibles en el sistema, como Crear Proyecto, Invitar Usuario, etc.
- **organization_role_permissions**: Tabla intermedia que asocia los roles de organización con sus permisos correspondientes y su nivel sobre estos permisos.
- **user_readmodels**: Almacena información de lectura específica de los usuarios dentro del contexto de una organización. Sigue el patrón Command Query Responsibility Segregation (CQRS)[8] que se detallará más adelante en el diseño de la arquitectura.

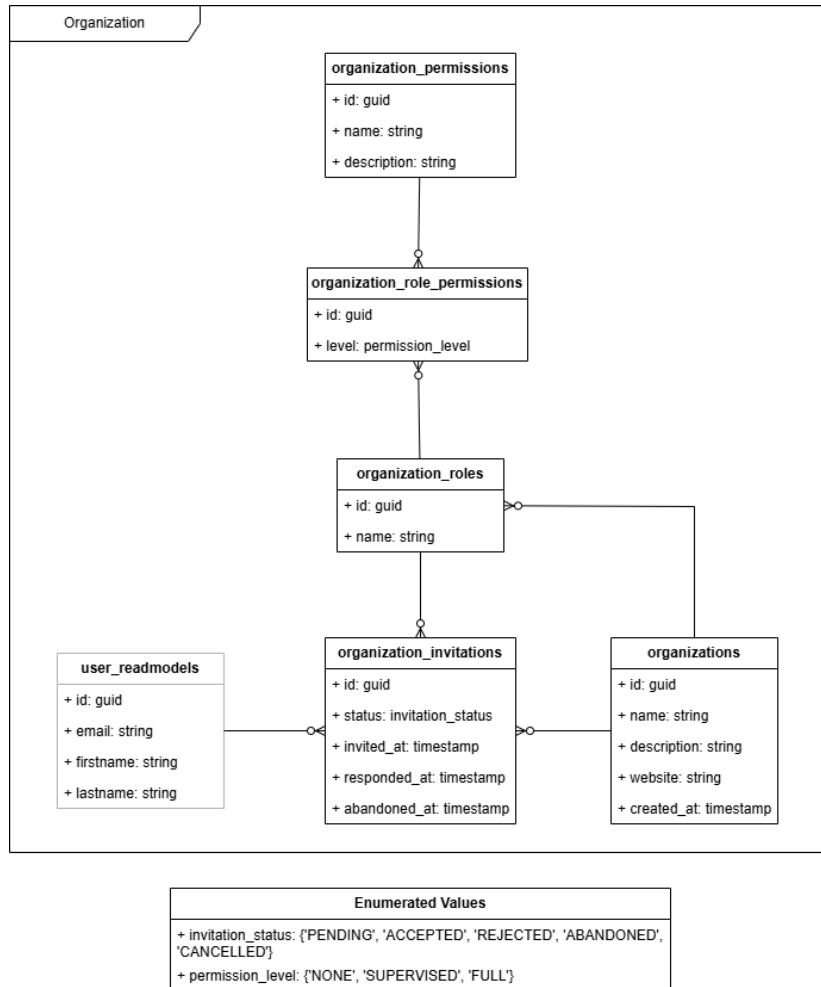


Figura 3.2: MER del módulo de Organizaciones

3.4.5 Módulo de Proyectos

Este módulo gestiona la información relacionada con los proyectos, sus miembros, roles, elementos de trabajo y otras configuraciones específicas de cada proyecto. Incluye las entidades:

- **projects:** Almacena la información básica de los proyectos, como su nombre, abreviación, privacidad, etc.
- **organization_readmodels:** Almacena información de lectura de las organizaciones para conocer la relación entre proyectos y organizaciones.
- **user_readmodels:** Almacena información de lectura de los usuarios para conocer su relación con los proyectos. Aquí encontramos un polimorfismo ya que un proyecto puede pertenecer a una organización o a un usuario de forma independiente.
- **agent_readmodels:** Almacena información de lectura de los agentes de IA para conocer los pertenecientes a un proyecto y su rol dentro de este.

- ***project_invitations***: Almacena las invitaciones para unirse a un proyecto, los miembros activos de un proyecto son los que tienen una invitación en estado aceptada. Aquí también encontramos un polimorfismo ya que se pueden invitar tanto usuarios humanos como agentes de IA.
- ***project_roles***: Almacena los roles definidos a nivel de proyecto, como Gestor, Desarrollador, etc.
- ***project_permissions_groups***: Almacena los grupos de permisos de proyecto disponibles en el sistema, únicamente se han definido para diferenciar visualmente los permisos que afectan a la gestión del proyecto de los que afectan a la gestión de los elementos de trabajo.
- ***project_permissions***: Almacena los permisos de proyecto disponibles en el sistema, como Configurar Proyecto, Crear Elemento de Trabajo, etc.
- ***project_role_permissions***: Tabla intermedia que asocia los roles de proyecto con sus permisos correspondientes y su nivel sobre estos permisos.
- ***field_types***: Almacena los tipos de campos personalizados disponibles en el sistema, como texto, número, fecha, etc.
- ***fields***: Almacena los campos personalizados de los proyectos, incluyendo su tipo y nombre.
- ***project_fields***: Tabla intermedia que asocia los campos personalizados con los proyectos a los que pertenecen.
- ***options***: Almacena las opciones de los campos personalizados de tipo selección, incluyendo su valor y su contenido.
- ***work_item_type_readmodels***: Almacena información de lectura de los tipos de elementos de trabajo, para poder relacionarlos con los campos personalizados.
- ***field_applies***: Tabla intermedia que asocia los campos personalizados con los tipos de elementos de trabajo a los que aplican.

- ***field_readmodels* y *field_type_readmodels*:** Almacenan información de lectura de los campos personalizados y sus tipos para conocer los campos personalizados que tiene un elemento de trabajo.
- ***field_values*:** Almacena los valores de los campos personalizados de cada elemento de trabajo.
- ***work_item_status*:** Almacena los estados de los elementos de trabajo definidos en el sistema.
- ***work_item_types*:** Almacena los tipos de elementos de trabajo definidos en el sistema.
- ***user_readmodels*:** Almacena información de lectura de los usuarios para conocer quién tiene asignado un elemento de trabajo y otros datos como quién hace comentarios o registra trabajo realizado.
- ***attachments*:** Almacena la información de los archivos adjuntos a los elementos de trabajo ya sea en su descripción o en los comentarios, incluyendo su URL y metadatos asociados.
- ***work_item_comments*:** Almacena los comentarios realizados en los elementos de trabajo, incluyendo el autor, el contenido y la fecha de creación.
- ***work_logs*:** Almacena los registros de trabajo realizados en los elementos de trabajo, incluyendo el usuario que realizó el registro, la cantidad de tiempo registrado y la fecha del registro.
- ***activity_logs*:** Almacena un historial de cambios y eventos relacionados con los elementos de trabajo, como cambios de estado, reasignaciones, actualizaciones de campos, etc.

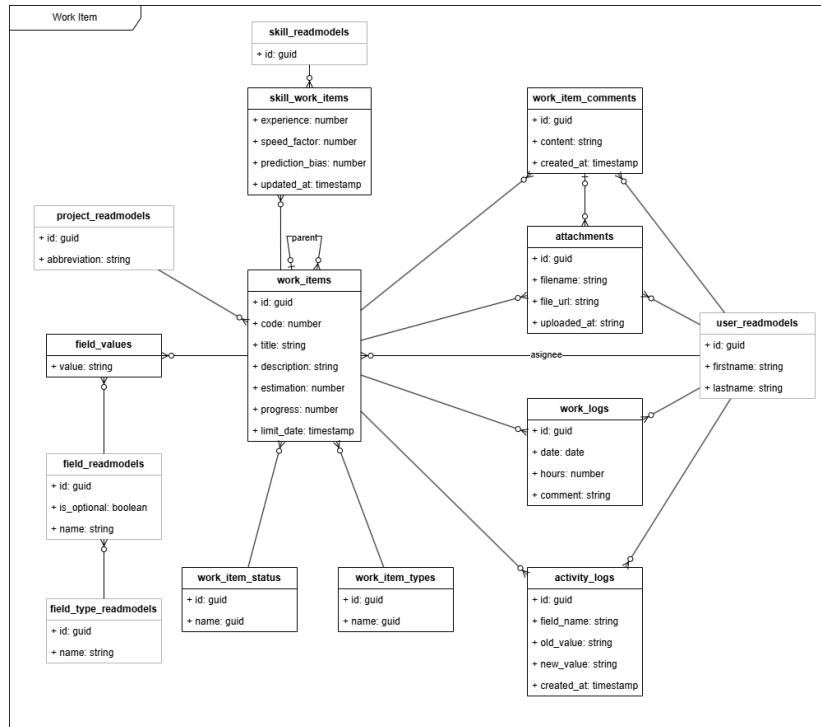


Figura 3.4: MER del módulo de Elementos de Trabajo

3.4.7 Módulo de Chats

Este módulo gestiona la información relacionada con los chats entre usuarios y agentes de IA. Incluye las entidades:

- **chats:** Almacena la información básica de los chats, como su identificador y fecha de creación.
- **chat_participants:** Tabla intermedia que asocia los chats con sus participantes, ya sean usuarios humanos o agentes de IA.
- **chat_messages:** Almacena los mensajes enviados en los chats, incluyendo el autor, el contenido, la fecha de envío y su estado (enviado, entregado, leído).
- **user_readmodels:** Almacena información de lectura de los usuarios para conocer su participación en los chats.
- **agent_readmodels:** Almacena información de lectura de los agentes de IA para conocer su participación en los chats.
- **project_readmodels:** Almacena información de lectura de los proyectos para conocer a qué proyecto pertenece un agente.

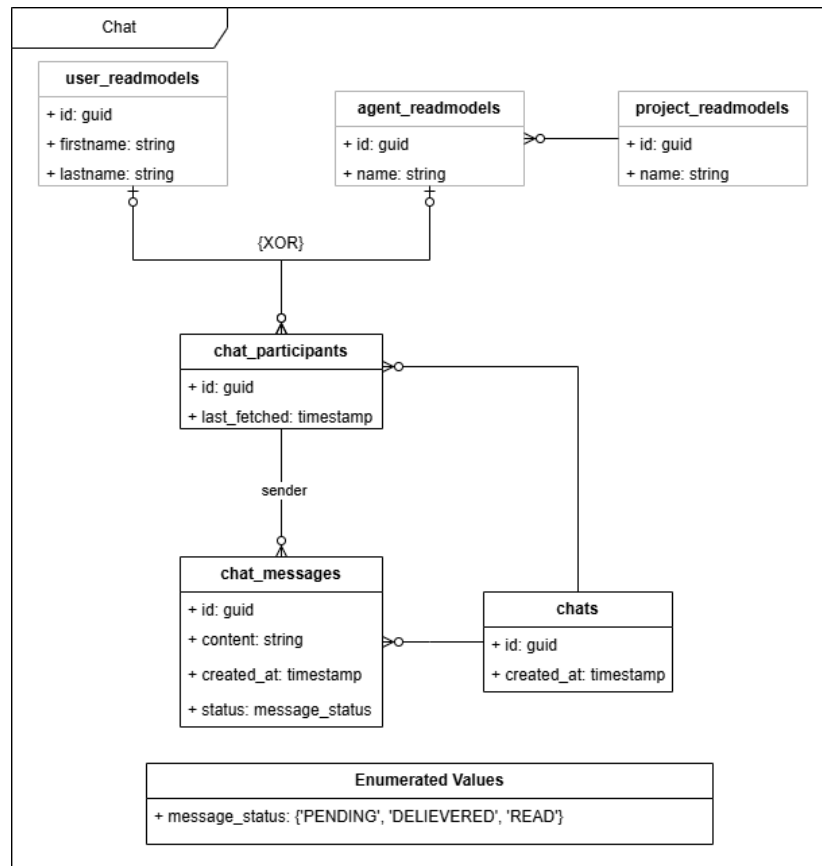


Figura 3.5: MER del módulo de Chats

3.4.8 Módulo de Agentes

Este módulo gestiona la información relacionada con los agentes de IA que colaboran en los proyectos. Incluye las entidades:

- **project_readmodels**: Almacena información de lectura de los proyectos para conocer a qué proyecto pertenece un agente.
- **model_providers**: Almacena los proveedores de modelos de IA disponibles en el sistema, como OpenAI, Claude, etc. junto con sus tokens de Interfaz de Programación de Aplicaciones (API).
- **models**: Almacena los modelos de IA disponibles en el sistema, asociados a un proveedor de modelos con su descripción y precio por millón de tokens de entrada.
- **agents**: Almacena la información de los agentes de IA creados en el sistema, incluyendo su nombre, instrucciones y *system prompt*.
- **model_configs**: Tabla intermedia que asocia los agentes de IA con los modelos de IA que utilizan, incluyendo la configuración específica del modelo (temperatura y máximo de tokens).

- ***agent_templates***: Almacena las plantillas de agentes de IA predefinidos en el sistema, incluyendo su nombre, descripción e instrucciones. Ejemplos de plantillas pueden ser el agente *Daily Standup* o el agente *Desglose de Trabajo*.
- ***agent_time_triggers***: Almacena los disparadores temporales configurados para los agentes de IA, incluyendo la frecuencia de ejecución, zona horaria etc. Estos disparadores permiten configurar la ejecución automática de los agentes en función del tiempo, como por ejemplo ejecutar el agente *Daily Standup* todos los días a las 9:00 am.
- ***agent_event_triggers***: Almacena los disparadores de eventos configurados para los agentes de IA, incluyendo el tipo de evento que los dispara (creación de elemento de trabajo, cambio de estado, etc.) y las condiciones específicas del evento. Estos disparadores permiten configurar la ejecución automática de los agentes en función de eventos específicos dentro del proyecto.
- ***agent_executions***: Representa una instancia de ejecución de un agente de IA, que puede ser disparada mediante alguno de los disparadores definidos (mode: regular) o mediante un mensaje directo de un usuario (mode: direct query). Almacena información sobre la ejecución del agente, como su estado (pendiente, en progreso, completada, fallida y suspendida) y el *payload* de entrada.
- ***agent_execution_messages***: Almacena los mensajes generados durante la ejecución de un agente de IA, incluyendo los mensajes del sistema, los mensajes de usuario y los mensajes generados por el agente. Estos mensajes permiten mantener un contexto completo de la ejecución del agente.
- ***agent_execution_pending_events***: Almacena los eventos de los que está pendiente una ejecución de un agente de IA para poder reactivar la ejecución cuando se cumplan las condiciones del evento. Por ejemplo, si la ejecución de un agente está pendiente de la respuesta de un usuario para acabar su ejecución. Es importante no solo el tipo de evento esperado, sino el *correlation_id* del evento para poder asociarlo con la ejecución correcta.

3. ANÁLISIS

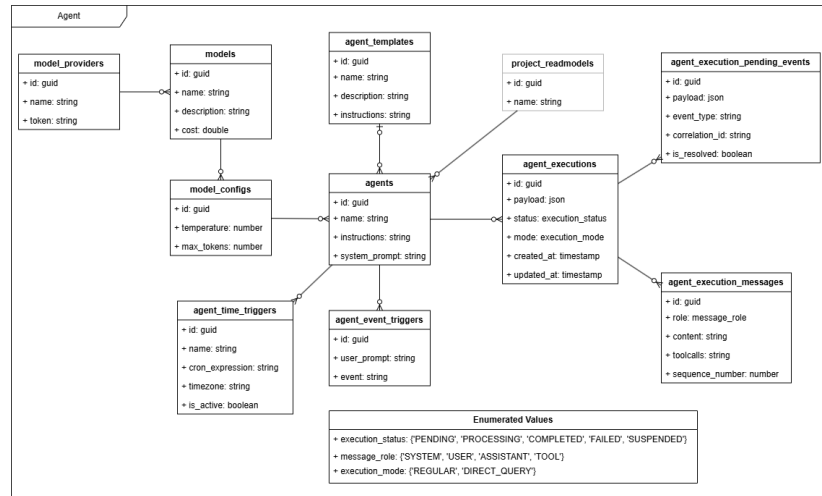


Figura 3.6: MER del módulo de Agentes

3.4.9 Modelo de Notificaciones

TODO

4.1 Arquitectura de la aplicación

Para representar la arquitectura de la aplicación, se ha recurrido a diagramas C4 [9], que permiten describir el sistema a diferentes niveles de abstracción, desde una visión general hasta los detalles de implementación. A continuación, se detallarán los tres primeros niveles de este modelo, llegando hasta el diagrama de componentes.

4.1.1 Diagrama de contexto (Nivel 1)

4.1.2 Diagrama de contenedores (Nivel 2)

4.1.3 Diagrama de componentes (Nivel 3)

Backend

Backend (dentro de un módulo)

Frontend

PLANIFICACIÓN

5.1 Alcance del proyecto

5.1.1 Definición del alcance

Una aplicación de gestión de proyectos profesional a día de hoy es una herramienta con muchísimas funcionalidades, y es imposible abarcar todo el espectro de posibilidades en un proyecto de estas características, tanto en fecha como en dedicación.

Por ello, es necesario definir un alcance claro y realista para el proyecto, que permita centrarse en las funcionalidades indispensables en aplicaciones de este tipo, añadiendo la innovación planteada en el proyecto, y dejando de lado funcionalidades que, aunque interesantes, no son imprescindibles para el correcto funcionamiento de la aplicación.

Para definir el alcance del proyecto y priorizar las funcionalidades desde el punto de vista del usuario final, se ha utilizado la técnica de las Historias de Usuario (User Stories), que permite describir las funcionalidades desde la perspectiva del usuario, y priorizarlas en función de su importancia para el usuario final. Para ello, el primer paso es identificar a los usuarios que van a utilizar la aplicación.

5.1.2 Exclusiones

Tras recopilar las historias de usuario, se han identificado algunas funcionalidades que, aunque podrían ser interesantes, quedan fuera del alcance del proyecto por no ser imprescindibles para el correcto funcionamiento de la aplicación, o por requerir un esfuerzo de desarrollo desproporcionado en relación al beneficio que aportan. De esta forma, se han desarrollado las historias de usuario con una prioridad de *Must Have* y *Should Have*, dejando fuera funcionalidades con prioridad de *Could Have* o *Won't Have*.

Además, por lo referente a la autenticación, se ha decidido limitar el proyecto a la autenticación mediante OAuth con Google, dejando fuera otras opciones de autenticación.

ción (Microsoft, Facebook, X, etc.) ya que el foco del proyecto no es la integración de múltiples proveedores de autenticación.

También se han dejado fuera funcionalidades relacionadas con la gestión de recursos (gestión de archivos, integración con repositorios de código, etc.) que aunque serían muy útiles e interesantes, no pueden ser implementadas dentro del alcance del proyecto.

5.2 Cronograma



ANEXOS

Texto placeholder de anexos. Aquí se añadirá material complementario cuando la memoria esté más avanzada.

BIBLIOGRAFÍA

- [1] D. Hardt, “The oauth 2.0 authorization framework,” RFC Editor, RFC 6749, 2012.
- [2] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [3] J. Nielsen and R. Molich, “Heuristic evaluation of user interfaces,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. ACM, 1990, pp. 249–256.
- [4] W3C World Wide Web Consortium, “Web content accessibility guidelines (wcag) 2.1,” <https://www.w3.org/TR/WCAG21/>, 2018.
- [5] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2004.
- [6] M. Fowler, “Bounded context,” <https://martinfowler.com/bliki/BoundedContext.html>, 2014, accedido: 11 de junio de 2026.
- [7] P. P.-S. Chen, “The entity-relationship model: Toward a unified view of data,” *ACM Transactions on Database Systems (TODS)*, vol. 1, no. 1, pp. 9–36, 1976.
- [8] M. Fowler, “Cqrs (command query responsibility segregation),” <https://martinfowler.com/bliki/CQRS.html>, 2011, accedido: 9 de junio de 2026.
- [9] S. Brown, “The c4 model for visualising software architecture,” <https://c4model.com/>, 2026, accedido: 10 de junio de 2026.